

Oddify

Arquitetura, ferramentas e convenções de desenvolvimento

01 Arquitetura geral

O Oddify é estruturado como **Modular Monolith** em .NET: uma única aplicação hospeda quatro módulos de negócio independentes — **Fixtures**, **Análise**, **Apostas** e **Users** — cada um com seu próprio schema no mesmo banco Postgres. Cada módulo é dividido em camadas (Domain, Application, Infrastructure, Presentation), com as dependências fluindo sempre de fora para dentro.

02 CQRS e DDD tático

É utilizado **CQRS**: comandos escrevem via EF Core, queries leem via Dapper puro — nunca EF do lado de leitura. As entidades seguem **DDD tático**: construtor privado, propriedades só alteráveis por métodos de comportamento, e eventos de domínio disparados somente quando o estado muda de fato.

03 Tratamento de erros

Erros de negócio são tratados com um padrão **Result / Result<T>**, sem lançar exceções para falhas esperadas. Exceções ficam reservadas apenas para casos realmente inesperados.

04 Comunicação entre módulos

A comunicação entre módulos é assíncrona, via eventos de integração (**MassTransit**): um módulo nunca acessa o código interno de outro diretamente — apenas reage a eventos publicados.

05 Princípios de código

É aplicado bastante **DRY**: DTOs reaproveitados entre queries, lógica repetitiva (validação, logging, tratamento de erro) centralizada em pipelines do MediatR ao invés de duplicada em cada handler. Os princípios **SOLID** também são seguidos de forma consistente:

- **Responsabilidade única** — cada handler resolve exatamente um caso de uso
- **Inversão de dependência** — interfaces para cache, banco e eventos, implementadas na Infrastructure
- **Repository + Unit of Work** — acesso a dados isolado do domínio
- **Convention over Configuration** — endpoints e handlers descobertos automaticamente, sem registro manual

06 Stack tecnica

- **.NET 10** — Minimal APIs, sem controllers
- **MediatR + FluentValidation** — orquestracao de comandos/queries e validacao
- **EF Core + Npgsql** — persistencia de escrita
- **Dapper** — leitura otimizada
- **MassTransit** — mensageria entre modulos
- **Redis** — cache, com fallback para memoria
- **Serilog + Seq** — logging estruturado
- **Swagger + JWT** — documentacao de API e autenticacao

07 Modelo de analise proprio

O modulo **Analise** implementa um modelo estatistico desenvolvido internamente, baseado em **Poisson-Dixon-Coles**, para calcular probabilidades de mercados esportivos e identificar oportunidades de aposta com valor esperado positivo. Nao e uma integracao externa — e logica de dominio propria do projeto.

08 Integracoes externas

- **API-Football e The Odds API** — dados esportivos e cotacoes

09 Qualidade e testes

O build utiliza **TreatWarningsAsErrors** ativado, analise estatica via SonarAnalyzer e um **.editorconfig** rigido. Os testes seguem o modelo **xUnit** estruturado em **Arrange-Act-Assert** (setup, chamada do metodo, depois os asserts, sem comentarios separando as secoes), com nomenclatura fixa **MetodoOuCenario_should_resultadoEsperado_when_condicao** (ex.: *RegistrarResultado_should_fail_when_partida_already_encerrada*). Casos com multiplas variacoes de entrada usam **[Theory]** + **[InlineData]** ao inves de duplicar **[Fact]**s. Tres niveis sao cobertos:

- **Testes unitarios** — xUnit + FluentAssertions (asserts fluentes) + NSubstitute (dependencias mockadas via Substitute.For<T>) — isolam entidades de dominio e handlers de comando/query
- **Testes de integracao** — xUnit + Microsoft.AspNetCore.Mvc.Testing + Testcontainers.PostgreSql + Respawn — sobem a API real (WebApplicationFactory) contra um Postgres em container via uma Collection Fixture compartilhada, resetando o banco entre os testes
- **Testes de arquitetura** — xUnit + NetArchTest.Rules — verificam automaticamente as regras de camada e fronteira entre modulos (ex.: Domain nao pode depender de Application/Infrastructure)

10 Infraestrutura local

A infraestrutura local roda via **Docker Compose**, com containers de Postgres, Redis e Seq.